[SVT Lab] Final Report: Testing Laboratory

Reza Ahmadi (VR510067)

May 21, 2026

1 Introduction

This report provides an overview and my personal understanding of the concepts covered in the Testing Laboratory of the System Verification and Testing (SVT) course. To ensure a comprehensive understanding, I have organized the report into several sections by the same order of the lectures.

2 Verification vs Testing

The first and the most important concept in the lab is the difference between verification and testing. In fact, verification is a process that ensures that the design meets the specified requirements and designers intentions, while testing is a process that evaluates the functionality and performance of a design by executing it with specific inputs and observing the outputs. In other words, we do the verification process before any physical implementation to test the design's correctness, while testing is done after the physical implementation to evaluate the design's behavior under real-world conditions. We can also say we have just one verification process (with multiple repeats) but we have a dedicated testing process for each created device. Another difference of the Verification and Testing is related to approach of running Verification and Testing. The verification process usually runs by simulators and emulators, while testing is done by using real hardware or test equipment. For instance, by considering manufacturing of an IC, after designing the IC in the design phase within computer software, we run the verification process by using simulators and emulators to check if the design is correct and meets the requirements, and we can repeat this process several times until we are sure about the design. At the end when we are sure about the design, we can start the manufacturing process. After manufacturing, we run the testing process by using real hardware or test equipment to evaluate the functionality and performance of the manufactured IC under real-world conditions.

3 Why Testing is Important?

One of the most important reasons for running testing steps after manufacturing is reducing the cost of faults. For instance, if we consider an IC, finding a fault at a lower level (wafer level) is simpler and cheaper than when it is used in a smartphone or a car. In fact, if we find a fault in the design phase, we can fix it before manufacturing, which is much cheaper than finding and fixing it after manufacturing. On the other hand it is understandable that testing helps to ensure about the quality of the product.

4 Verilog-AMS

Verilog-AMS is a hardware description language that helps us for modeling and verification of mixed-signal systems. Traditional testing as we discussed was on the manufactured devices but Verilog-AMS allows us to pre-silicon verify by simulating the behavior

of analog and digital components before fabrication. By the presented samples that we have discussed on the relevent course, From the code examples presented in the lecture, several key aspects of Verilog-AMS can be understood. Here I am explaining the most important ones.

4.1 Modular Structure

In the Verilog-AMS every ***component*** has its own ***module***, and every module has an interface with input and output ports, internal parameters and behavior section which describe how it works. And this feature allows us to have an easy reusable in large-scale design. The modularity is very useful in implementing complex and large systems. For example in the DC motor model, the electrical winding and the mechanical rotor are described separately but connected through shared nodes. This separation keeps the code clean and easy to understand.

4.2 Branch Contribution ($<+$)

The $<+$ operator is one of the most important concepts in Verilog-AMS. It uses for describe the physical relationship between two nodes, such as, the voltage across a resistor or the current through a capacitor. For example this equation $V(p,n) <+ r \cdot I(p,n)$ is simply Ohm's law written in code form. This approach allows the analog solver uses these contributions to build and solve a system of equations automatically. The engineers only need to describe the physics, and the simulator handles the mathematics behind finding voltages and currents at every node.

4.3 Disciplines

In Verilog there are a categorized system that is called Disciplines, there are multiple types of discipline such as *electrical*, *thermal*, *rotational*. The discipline tells the simulator what kind of physical quantity the signal represents, for example, electrical signals have a voltage as potential and current as flow. These Disciplines are really useful in some specified implementation, For instance in the DC motor example we have seen a module has nodes (electrical and rotational). Without disciplines, the simulator would not know how to handle different types of signals in the same circuit.

5 The Fault characteristic

In simple words the fault is when a part of circuit does not work properly, In fact we have discussed that when a part of circuit like a transistor or another electronic part has been corrupted by a dust in metal layout or another reason and it affects on the functionality of the circuit and the fault occurs. We have single and multiple faults that indicates complexity. Furthermore faults are categorized in different types such as ***Electrical*** and ***Mechanical***.

5.1 Analog Fault

Analog faults happen when a physical component changes its behavior, for example two wires should be separated but accidentally they touch each other, and this connection changes the behavior of the circuit(Also this specific way of fault called Bridge Fault).

5.2 Digital Fault

Digital faults occur when a wire or logic gate gets stuck, we have discussed that due to binary environment of the circuit it will be stuck on 0 or 1 all the time.

5.3 Analog Fault Vs Digital Fault

In simple words we can say an analog fault represents a wrong behavior (dynamically changing) but a digital fault represents

wrong values (0 instead of 1 or 1 instead of 0).

5.4 Digital Fault Models

The Stuck-at model is one of the most important concepts in Digital-Faults. It uses for describe the logical relationship between two nodes, such as, the zero across a wire or the one through a gate. Delay faults happen when a physical gate changes its timing, for example two signals should be synchronized but accidentally they miss each other, and this delay changes the behavior of the circuit(Also this specific way of fault called Transition Fault). In simple words we can say an observability concept represents a read behavior (externally monitoring) but a controllability concept represents write values (1 instead of 0 or 0 instead of 1).

5.5 IDDQ Testing

One of the most important reasons for running current measurements during manufacturing is reducing the leakage of faults. For instance, if we consider an IC, finding a fault at a quiescent state (IDDQ state) is simpler and cheaper than when it is detected in a functional test or a system. In fact, if we find a fault in the layout phase, we can fix it before fabrication, which is much cheaper than finding and fixing it after fabrication. On the other hand it is understandable that IDDQ helps to ensure about the reliability of the product.

6 ATPG

Automatic Test Pattern Generation is a software tool that helps designers to generate all the possible inputs of the circuit/chip automatically and without human handling needed. In this approach there are several advantages, the most important one is related to almost being guaranteed that we have all the possible inputs for the circuit/chip and we can test all shapes of order on it. Also, it reduces human interaction and costs. ATPG works on the Gate-Level layout. ATPG approaches are separated into two categories, Combinational and Sequential. Combinationals are related to memoryless and combinational circuits, but Sequentials are sequential circuits (with flip-flops). Sequential circuits are much harder to test because of internal states. Tessent FastScan is a tool that is introduced in the lecture.

6.1 Fault Coverage

The result of running ATPG is Fault Coverage; we measure what percentage of all possible faults were detected. This is called fault coverage, and the goal is to get as close to 100% as possible.

6.2 Fault Simulation and Yield

Fault Simulation is a software tool that helps designers to evaluate all the possible faults of the circuit/chip automatically and without human handling needed. In fact, ATPG is a process that ensures that the design receives the specified inputs and externally generated vectors, while BIST is a process that evaluates the functionality and performance of a design by executing it with internal inputs and observing the signatures. In this approach there are several advantages, the most important one is related to almost being guaranteed that we have all the internal generations for the circuit/chip and we can test all speeds of clock on it. At the end when we measure what percentage of all manufactured chips were defect-free. This is called manufacturing yield, and the goal is to get as close to 100% as possible.

7 Design for Testability (DFT)

One of the most important reasons for implementing design for testability before manufacturing is improving the controllability of

faults. In fact, if we add a scan-chain in the design phase, we can test it after manufacturing, which is much cheaper than finding and testing it without scan-chains. DFT approaches are separated into two categories, Ad-hoc and Structured. Ad-hocs are related to specific and localized circuits, but Structureds are systematic circuits (with scan-paths). Scan chains are much easier to test because of shifted states. The scan-chain is very useful in testing complex and sequential systems.

7.1 Built-In Self-Test (BIST)

Built-In Self-Test is a hardware architecture that helps designers to generate all the possible vectors of the circuit/chip automatically and without external equipment needed. In this approach there are several advantages, the most important one is related to almost being guaranteed that we have all the possible vectors for the circuit/chip and we can test all shapes of speed on it. Also, it reduces external interaction and costs. BIST works on the Board-Level layout. BIST approaches are separated into two categories, Online and Offline. Onlines are related to concurrent and operational circuits, but Offlines are non-concurrent circuits (with test-modes). Offline circuits are much easier to test because of isolated states. Tessent BISTRock is a tool that is introduced in the lecture.

7.2 BILBO Architecture

In the BILBO register every ***flip-flop*** has its own ***mode***, and every mode has an interface with scan and parallel ports, internal configurations and operation section which describe how it works. And this feature allows us to have an easy reusable in large-scale testing. The programmability is very useful in implementing complex and scan systems. For example in the signature analysis model, the pseudo-random generation and the parallel signature are described separately but connected through shared registers. This separation keeps the code clean and easy to understand.

8 Boundary Scan Standard

Boundary Scan is a structural testing standard that helps us for modeling and verification of board-level systems. Traditional testing as we discussed was on the manufactured devices but Boundary Scan allows us to post-silicon verify by controlling the behavior of input and output pins before packaging. By the presented samples that we have discussed on the relevent course, From the architecture examples presented in the lecture, several key aspects of Boundary Scan can be understood. Here I am explaining the most important ones.